

- Estos algoritmos ordenan arreglos comparando los elementos, es decir, son algoritmos de sorting por comparación.
- Vimos en la teórica que en el peor caso son $\Omega(n \log n)$.
- ¿Puede haber algo mejor?
- Sí, por ejemplo, si tenemos información de los elementos.
- O podríamos tener complejidades sujetas a otros factores, más allá del tamaño de entrada.

Bucket Sort

- **Bucket Sort** ordena arreglos de cualquier tipo.
- Supone que los elementos pueden separarse (según algún criterio) en M categorías ordenadas.
- Es decir, para $i < j$, todo elemento de la categoría i es menor que todo elemento de la categoría j .

Bucket Sort

- **Bucket Sort** ordena arreglos de cualquier tipo.
- Supone que los elementos pueden separarse (según algún criterio) en M categorías ordenadas.
- Es decir, para $i < j$, todo elemento de la categoría i es menor que todo elemento de la categoría j .
- Idea:
 - 1 Construir un arreglo B de M listas y guardar los elementos de la categoría i en la i -ésima lista.
 - 2 Ordenar las M listas por separado (si restara algo por ordenar).
 - 3 Reconstruir el arreglo A , concatenando las listas en orden.

- $h(x)$ es una función que indica la categoría de x (de 0 a $M - 1$).
- Suponemos que se ejecuta en $O(1)$.

Algorithm 1 BUCKET-SORT(A, M)

```
 $n \leftarrow \text{length}(A)$   
 $B \leftarrow$  arreglo de  $M$  listas vacías  
for  $i \leftarrow [0..n - 1]$  do  
    insertar  $A[i]$  en la lista  $B[h(A[i])]$   
end for  
for  $j \leftarrow [0..M - 1]$  do  
    ordenar lista  $B[j]$   
end for  
concatenar listas  $B[0], B[1], \dots, B[M - 1]$ 
```

- ¿Complejidad?

Bucket Sort

- $h(x)$ es una función que indica la categoría de x (de 0 a $M - 1$).
- Suponemos que se ejecuta en $O(1)$.

Algorithm 1 BUCKET-SORT(A, M)

```
 $n \leftarrow \text{length}(A)$   
 $B \leftarrow$  arreglo de  $M$  listas vacías  
for  $i \leftarrow [0..n - 1]$  do  
    insertar  $A[i]$  en la lista  $B[h(A[i])]$   
end for  
for  $j \leftarrow [0..M - 1]$  do  
    ordenar lista  $B[j]$   
end for  
concatenar listas  $B[0], B[1], \dots, B[M - 1]$ 
```

- ¿Complejidad? $O(n + M) + O(\text{ordenar buckets})$, con $n = \text{length}(A)$.

Bucket Sort

- $h(x)$ es una función que indica la categoría de x (de 0 a $M - 1$).
- Suponemos que se ejecuta en $O(1)$.

Algorithm 1 BUCKET-SORT(A, M)

```
 $n \leftarrow \text{length}(A)$   
 $B \leftarrow$  arreglo de  $M$  listas vacías  
for  $i \leftarrow [0..n - 1]$  do  
    insertar  $A[i]$  en la lista  $B[h(A[i])]$   
end for  
for  $j \leftarrow [0..M - 1]$  do  
    ordenar lista  $B[j]$   
end for  
concatenar listas  $B[0], B[1], \dots, B[M - 1]$ 
```

- ¿Complejidad? $O(n + M) + O(\text{ordenar buckets})$, con $n = \text{length}(A)$.
- Si se omite el paso 2... $O(n + M)$.

Counting Sort

- **Counting Sort** asume que los elementos de un arreglo A de naturales **son menores** que un cierto valor k .

Counting Sort

- **Counting Sort** asume que los elementos de un arreglo A de naturales **son menores** que un cierto valor k .
- Idea:
 - 1 Construir un arreglo B de tamaño k y guardar en la posición i , la cantidad de apariciones de i en A .
 - 2 Reconstruir el arreglo A , poniendo tantas copias de cada valor $j \in \{0..k\}$, como lo indique $B[j]$.

Counting Sort

- **Counting Sort** asume que los elementos de un arreglo A de naturales **son menores** que un cierto valor k .
- Idea:
 - 1 Construir un arreglo B de tamaño k y guardar en la posición i , la cantidad de apariciones de i en A .
 - 2 Reconstruir el arreglo A , poniendo tantas copias de cada valor $j \in \{0..k\}$, como lo indique $B[j]$.
- **Observación:** Para un arreglo cualquiera de naturales, puede tomarse $k = \max_i \{A[i]\} + 1$.

Counting Sort

- **Counting Sort** asume que los elementos de un arreglo A de naturales **son menores** que un cierto valor k .
- Idea:
 - 1 Construir un arreglo B de tamaño k y guardar en la posición i , la cantidad de apariciones de i en A .
 - 2 Reconstruir el arreglo A , poniendo tantas copias de cada valor $j \in \{0..k\}$, como lo indique $B[j]$.
- **Observación:** Para un arreglo cualquiera de naturales, puede tomarse $k = \max_i \{A[i]\} + 1$.
- **Observación 2:** Se puede adaptar para el caso en que los valores están contenidos en un intervalo $[d, d + k)$.

Counting Sort

Precondición: $k > \max_i \{A[i]\}$

Algorithm 2 COUNTING-SORT(A : arreglo(nat), k : nat)

```
1:  $B \leftarrow$  arreglo de tamaño  $k$ 
2: for  $i \leftarrow [0..k - 1]$  do
3:    $B[i] \leftarrow 0$ 
4: end for
5: //cuento la cantidad de apariciones de cada elemento.
6: for  $j \leftarrow [0..length(A) - 1]$  do
7:    $B[A[j]] \leftarrow B[A[j]] + 1$ 
8: end for
9: //inserto en A la cantidad de apariciones de cada elemento.
10:  $indexA \leftarrow 0$ 
11: for  $i \leftarrow [0..k - 1]$  do
12:   for  $j \leftarrow [1..B[i]]$  do
13:      $A[indexA] \leftarrow i$ 
14:      $indexA \leftarrow indexA + 1$ 
15:   end for
16: end for
```

Counting Sort

Precondición: $k > \max_i \{A[i]\}$

Algorithm 2 COUNTING-SORT(A : arreglo(nat), k : nat)

```
1:  $B \leftarrow$  arreglo de tamaño  $k$ 
2: for  $i \leftarrow [0..k - 1]$  do
3:    $B[i] \leftarrow 0$ 
4: end for
5: //cuento la cantidad de apariciones de cada elemento.
6: for  $j \leftarrow [0..length(A) - 1]$  do
7:    $B[A[j]] \leftarrow B[A[j]] + 1$ 
8: end for
9: //inserto en A la cantidad de apariciones de cada elemento.
10:  $indexA \leftarrow 0$ 
11: for  $i \leftarrow [0..k - 1]$  do
12:   for  $j \leftarrow [1..B[i]]$  do
13:      $A[indexA] \leftarrow i$ 
14:      $indexA \leftarrow indexA + 1$ 
15:   end for
16: end for
```

- ¿Complejidad?

Counting Sort

Precondición: $k > \max_i \{A[i]\}$

Algorithm 2 COUNTING-SORT(A : arreglo(nat), k : nat)

```
1:  $B \leftarrow$  arreglo de tamaño  $k$ 
2: for  $i \leftarrow [0..k - 1]$  do
3:    $B[i] \leftarrow 0$ 
4: end for
5: //cuento la cantidad de apariciones de cada elemento.
6: for  $j \leftarrow [0..length(A) - 1]$  do
7:    $B[A[j]] \leftarrow B[A[j]] + 1$ 
8: end for
9: //inserto en A la cantidad de apariciones de cada elemento.
10:  $indexA \leftarrow 0$ 
11: for  $i \leftarrow [0..k - 1]$  do
12:   for  $j \leftarrow [1..B[i]]$  do
13:      $A[indexA] \leftarrow i$ 
14:      $indexA \leftarrow indexA + 1$ 
15:   end for
16: end for
```

- ¿Complejidad? $O(n + k)$, con $n = length(A)$

Counting Sort

Precondición: $k > \max_i \{A[i]\}$

Algorithm 2 COUNTING-SORT(A : arreglo(nat), k : nat)

```
1:  $B \leftarrow$  arreglo de tamaño  $k$ 
2: for  $i \leftarrow [0..k - 1]$  do
3:    $B[i] \leftarrow 0$ 
4: end for
5: //cuento la cantidad de apariciones de cada elemento.
6: for  $j \leftarrow [0..length(A) - 1]$  do
7:    $B[A[j]] \leftarrow B[A[j]] + 1$ 
8: end for
9: //inserto en A la cantidad de apariciones de cada elemento.
10:  $indexA \leftarrow 0$ 
11: for  $i \leftarrow [0..k - 1]$  do
12:   for  $j \leftarrow [1..B[i]]$  do
13:      $A[indexA] \leftarrow i$ 
14:      $indexA \leftarrow indexA + 1$ 
15:   end for
16: end for
```

- ¿Complejidad? $O(n + k)$, con $n = length(A)$
- En el Cormen hay otra versión (más complicada de entender pero más fácil de analizar su complejidad)

Radix Sort (versión para ordenar naturales)

- **Radix Sort**, en su versión más básica, ordena una lista de naturales.
- **Idea:**
 - 1 Ordenar los valores mirando sólo el dígito 1

Radix Sort (versión para ordenar naturales)

- **Radix Sort**, en su versión más básica, ordena una lista de naturales.
- **Idea:**
 - 1 Ordenar los valores mirando sólo el dígito 1
 - 2 Ordenar los valores mirando sólo el dígito 2 (manteniendo el orden en caso de empate)

Radix Sort (versión para ordenar naturales)

- **Radix Sort**, en su versión más básica, ordena una lista de naturales.
- **Idea:**
 - 1 Ordenar los valores mirando sólo el dígito 1
 - 2 Ordenar los valores mirando sólo el dígito 2
(manteniendo el orden en caso de empate)
 - 3 Ordenar los valores mirando sólo el dígito 3
(manteniendo el orden en caso de empate)

Radix Sort (versión para ordenar naturales)

- **Radix Sort**, en su versión más básica, ordena una lista de naturales.
- **Idea:**
 - 1 Ordenar los valores mirando sólo el dígito 1
 - 2 Ordenar los valores mirando sólo el dígito 2
(manteniendo el orden en caso de empate)
 - 3 Ordenar los valores mirando sólo el dígito 3
(manteniendo el orden en caso de empate)
 - 4 ...

Radix Sort (versión para ordenar naturales)

- **Radix Sort**, en su versión más básica, ordena una lista de naturales.

- **Idea:**

- 1 Ordenar los valores mirando sólo el dígito 1
- 2 Ordenar los valores mirando sólo el dígito 2
(manteniendo el orden en caso de empate)
- 3 Ordenar los valores mirando sólo el dígito 3
(manteniendo el orden en caso de empate)
- 4 ...

- **Ejemplo:**

329

457

657

839

436

720

355

Radix Sort (versión para ordenar naturales)

- **Radix Sort**, en su versión más básica, ordena una lista de naturales.

- **Idea:**

- 1 Ordenar los valores mirando sólo el dígito 1
- 2 Ordenar los valores mirando sólo el dígito 2 (manteniendo el orden en caso de empate)
- 3 Ordenar los valores mirando sólo el dígito 3 (manteniendo el orden en caso de empate)
- 4 ...

- **Ejemplo:**

329		720
457		355
657		436
839	⟶	457
436		467
720		329
355		839

Radix Sort (versión para ordenar naturales)

- **Radix Sort**, en su versión más básica, ordena una lista de naturales.

- **Idea:**

- 1 Ordenar los valores mirando sólo el dígito 1
- 2 Ordenar los valores mirando sólo el dígito 2 (manteniendo el orden en caso de empate)
- 3 Ordenar los valores mirando sólo el dígito 3 (manteniendo el orden en caso de empate)
- 4 ...

- **Ejemplo:**

329		720		720
457		355		329
657		436		436
839	↪	457	↪	839
436		467		355
720		329		457
355		839		657

Radix Sort (versión para ordenar naturales)

- **Radix Sort**, en su versión más básica, ordena una lista de naturales.

- **Idea:**

- 1 Ordenar los valores mirando sólo el dígito 1
- 2 Ordenar los valores mirando sólo el dígito 2 (manteniendo el orden en caso de empate)
- 3 Ordenar los valores mirando sólo el dígito 3 (manteniendo el orden en caso de empate)
- 4 ...

- **Ejemplo:**

329		720		720		329
457		355		329		355
657		436		436		436
839	→	457	→	839	→	457
436		467		355		657
720		329		457		720
355		839		657		839

Radix Sort (versión para ordenar naturales)

- Llamamos d a la cantidad máxima de dígitos de los elementos y suponemos que el dígito 1 es el menos significativo de cada valor.

Radix Sort (versión para ordenar naturales)

- Llamamos d a la cantidad máxima de dígitos de los elementos y suponemos que el dígito 1 es el menos significativo de cada valor.

Algorithm 3 RADIX-SORT(A, d)

```
for  $i \leftarrow [1..d]$  // del menos significativo al más significativo do  
    Ordenar el arreglo  $A$  según el dígito  $i$ , en forma estable  
end for
```

- Recordemos que un algoritmo es **estable** si preserva el orden original entre las cosas equivalentes.
- ¿Complejidad?

Radix Sort (versión para ordenar naturales)

- Llamamos d a la cantidad máxima de dígitos de los elementos y suponemos que el dígito 1 es el menos significativo de cada valor.

Algorithm 3 RADIX-SORT(A, d)

```
for  $i \leftarrow [1..d]$  // del menos significativo al más significativo do  
    Ordenar el arreglo  $A$  según el dígito  $i$ , en forma estable  
end for
```

- Recordemos que un algoritmo es **estable** si preserva el orden original entre las cosas equivalentes.
- ¿Complejidad? $d \times O(\text{ordenar } A \text{ por un dígito})$.
- ¿Con Bucket Sort (sin ordenar los buckets)?

Radix Sort (versión para ordenar naturales)

- Llamamos d a la cantidad máxima de dígitos de los elementos y suponemos que el dígito 1 es el menos significativo de cada valor.

Algorithm 3 RADIX-SORT(A, d)

```
for  $i \leftarrow [1..d]$  // del menos significativo al más significativo do  
    Ordenar el arreglo  $A$  según el dígito  $i$ , en forma estable  
end for
```

- Recordemos que un algoritmo es **estable** si preserva el orden original entre las cosas equivalentes.
- ¿Complejidad? $d \times O(\text{ordenar } A \text{ por un dígito})$.
- ¿Con Bucket Sort (sin ordenar los buckets)? $d \times O(n) = O(n \cdot d)$.

Radix Sort (versión para ordenar naturales)

- Llamamos d a la cantidad máxima de dígitos de los elementos y suponemos que el dígito 1 es el menos significativo de cada valor.

Algorithm 3 RADIX-SORT(A, d)

```
for  $i \leftarrow [1..d]$  // del menos significativo al más significativo do  
    Ordenar el arreglo  $A$  según el dígito  $i$ , en forma estable  
end for
```

- Recordemos que un algoritmo es **estable** si preserva el orden original entre las cosas equivalentes.
- ¿Complejidad? $d \times O(\text{ordenar } A \text{ por un dígito})$.
- ¿Con Bucket Sort (sin ordenar los buckets)? $d \times O(n) = O(n \cdot d)$.
- O sea... $O(n \cdot \log(\max(A)))$.

Radix Sort (generalizando)

- Radix Sort puede usarse también para ordenar cadenas de caracteres (la primera letra es la más significativa).

Radix Sort (generalizando)

- Radix Sort puede usarse también para ordenar cadenas de caracteres (la primera letra es la más significativa).
- El mismo esquema sirve para ordenar tuplas, donde el orden que se quiera dar depende principalmente de un “dígito” (i.e., un campo de la tupla) y en el caso de empate se tengan que revisar los otros.

Radix Sort (generalizando)

- Radix Sort puede usarse también para ordenar cadenas de caracteres (la primera letra es la más significativa).
- El mismo esquema sirve para ordenar tuplas, donde el orden que se quiera dar depende principalmente de un “dígito” (i.e., un campo de la tupla) y en el caso de empate se tengan que revisar los otros.
- La idea en el fondo es la misma: usar un **algoritmo estable** e ir ordenado por “dígito” (del menos al más significativo).

Algorithm 4 RADIX-SORT(A , d)

```
for  $i \leftarrow [1..d]$  // del menos significativo al más significativo do  
    Ordenar el arreglo  $A$  según el dígito  $i$ , en forma estable  
end for
```

- La complejidad en este caso es $d \times O(\text{ordenar } A \text{ por un dígito})$.

Algunas aclaraciones:

- Requieren práctica y paciencia.
- También requieren saber un poco de estructuras de datos.
- NO requieren que diseñen dichas estructuras (a menos que no sean las de siempre; como cuando hacen los ejercicios de elección de estructuras).
- Cada línea de código que escriban debería estar acompañada de la correspondiente complejidad temporal.

Algunas estrategias:

- Utilizar algoritmos ya conocidos (Merge Sort, Quick Sort, Counting Sort, Bucket Sort, Radix Sort, etc.)
- Utilizar estructuras de datos ya conocidas (AVL, Heap, Trie, listas enlazadas, etc.)
- Analizar la complejidad pedida e deducir algo de eso.
- Si conocemos algo de la entrada, ver cómo se puede usar para mejorar la complejidad.